Отчёт по 1 этапу курсовой работы

по дисциплине «Информационные системы»

Выполнили: Гладышев Иван
Хлопцов Сергей

Руководитель: Кобик Никита Алексеевич

2026 г.

1. Описание предметной области

1.1. Акторы предметной области

1. **Юрист / менеджер по продажам** - сотрудник юридической фирмы, который контактирует с клиентом (напрямую или через агента), проводит первичные переговоры, согласует стоимость услуг и доводит сделку до подписания договора.

2. **Системный администратор** - сотрудник компании, помагает юристу пользоваться платформой, проводит первоначальную настройку, обеспечивает поддержку.

1.2. Проблемы, которые будет решать информационная система:

1. Внутренние регламенты запрещают фирме вести дела против своих текущих или недавних клиентов. Например, если фирма ранее представляла интересы ПАО «Сбербанк» или его дочерних структур, она не может взять новое дело, где оппонентом выступает Сбербанк или любая связанная с ним компания.

   Какое решение есть сейчас: при каждом новом обращении юрист вручную проверяет историю фирмы по памяти, Excel-таблицам и внутренним архивам. Процесс занимает значительное время и не гарантирует полноты проверки, особенно в части установления связей «материнская компания - дочерняя структура».

2. Один и тот же клиент может обратиться к нескольким агентам одновременно, и каждый из них приведёт его в разные юридические фирмы (или к разным менеджерам внутри одной фирмы). В результате один и тот же лид обрабатывается несколькими менеджерами параллельно, что приводит к:

   - внутренним конфликтам между менеджерами;
   - необходимости вручную согласовывать, кто «первый» привёл клиента;
   - риску переплаты комиссии агентам.

   Кроме того, клиент может предложить разные финансовые условия разным агентам (например, через одного агента - 30 тыс. руб., через другого - 40 тыс. руб.), и фирме необходимо выбрать вариант с максимальной чистой прибылью (сумма клиента минус комиссия агента).

   Что сделано сейчас: менеджер вынужден опрашивать всех коллег в рабочих чатах («А к тебе ВТБ случайно не приходил?»). Ответы приходят не от всех (сотрудники в отпуске, не читают сообщения), информация теряется, решения принимаются на основе неполных данных.

3. Один и тот же фактический клиент может быть зарегистрирован в системе под разными именами. Типовой сценарий: муж и жена отдельно ищут юристов через разных агентов, но по факту их дело - одно и то же (например, раздел имущества). Без автоматического объединения фирма рискует:

   - вести параллельную работу по одному делу;
   - заплатить комиссию двум разным агентам за одного клиента;
   - получить конфликт интересов внутри собственной базы.

   Сейчас решение этой проблемы отсутствует. Менеджеры выявляют дубликаты только постфактум, на стадии передачи дела юристам.

4. Агент приводит клиента, но подписание договора может затянуться. Возникает ситуация, когда один агент «заморозил» клиента, но не может закрыть сделку, а другой агент - готов привести того же клиента быстрее и на более выгодных условиях.

   На практике было зафиксировано: первый агент потребовал комиссию, несмотря на то что договор подписал второй агент, мотивируя это тем, что «я привёл клиента первым». Фирме пришлось выплатить обе комиссии.

   На данный момент было внедрено правило - если агент не подписывает договор в течение 3 месяцев, он теряет приоритет. Однако правило применяется вручную и не фиксируется в системе.

2. Требования к информационной системе

2.1. Функциональные требования

№	Требование
FR-1	Система должна обеспечивать регистрацию и хранение данных о клиентах (физических и юридических лицах), агентах и менеджерах по продажам с привязкой к конкретной организации (юридической фирме).
FR-2	Система должна предоставлять возможность регистрации нового лида с указанием клиента, агента, оппонента и предполагаемых финансовых условий.
FR-3	Система должна автоматически выполнять проверку конфликта интересов при регистрации лида, сверяя оппонента с базой текущих и архивных клиентов организации.
FR-4	Система должна поддерживать проверку аффилированности компаний (связи «материнская - дочерняя» структуры) на основе глобального справочника.
FR-5	Система должна выявлять дубликаты клиентов по совпадающим или сходным реквизитам (ФИО, телефон, ИНН, адрес) и предлагать их объединение.
FR-6	Система должна выявлять случаи, когда один и тот же клиент приведён разными агентами, и отображать сводную таблицу финансовых предложений.
FR-7	Система должна автоматически рассчитывать чистую прибыль по каждому предложению агента и выделять наиболее выгодное.
FR-8	Система должна вести учёт времени нахождения лида в работе у конкретного агента и автоматически снимать приоритет по истечении заданного срока.

FR-9	Система должна предоставлять визуальный конструктор бизнес-правил (декларативное программирование), позволяющий администратору системы настраивать условия проверок, сроки и проценты без изменения исходного кода.
FR-10	Система должна обеспечивать архитектуру с полной изоляцией данных между разными юридическими фирмами.
FR-11	Система должна предоставлять разграничение ролей: дминистратор платформы, юрист (с ограниченным доступом).
FR-12	Система должна поддерживать интеграцию с внешними системами учёта рабочего времени (в частности, ProjectMate) через API.

2.2. Нефункциональные требования

№	Категория FURPS+	Требование
NFR-1	**Performance** (Производительность)	Время отклика системы при выполнении типовых операций (регистрация лида, проверка конфликта интересов) не должно превышать 2 секунд при одновременной работе до 100 пользователей.
NFR-2	+ **Security** (Безопасность)	Данные разных организаций должны быть строго изолированы. Доступ к данным организации возможен только для авторизованных пользователей этой организации.
NFR-3	+ **Scalability** (Масштабируемость)	Архитектура должна позволять добавление новых пользователей (юристов) без существенного снижения производительности.
NFR-4	**Reliability** (Надёжность)	Система должна обеспечивать доступность не менее 99% в рабочее время. Должна быть предусмотрена возможность восстановления данных после сбоев.
NFR-5	**Supportability** (Поддерживаемость)	Все бизнес-правила должны храниться в базе данных и применяться динамически, без перекомпиляции приложения. Это обеспечивает адаптируемость и лёгкость сопровождения системы.
NFR-6	**Supportability** (Поддерживаемость)	Система должна быть пригодна для развёртывания на helios, а также поддерживать контейнеризацию (Docker) для упрощения развёртывания в других средах.

NFR-7	**Usability** (Юзабилити)	Интерфейс конструктора правил должен быть доступен для пользователей без технического образования (визуальное программирование по типу Scratch). Документация системы должна быть интуитивно понятной.

3. Модели прецедентов

3.1. Акторы системы

В системе выделены следующие акторы:

- **Менеджер по продажам** — основной пользователь системы, регистрирует лиды, работает с клиентами и агентами.
- **Администратор организации** — владелец юридической фирмы, настраивает бизнес-правила, управляет пользователями своей организации.
- **Супер-администратор** — владелец b2b-платформы, управляет лицензиями, тарифами и глобальными настройками.
- **Система (автоматика)** — внутренний актор, выполняющий фоновые задачи: проверку конфликтов, дедупликацию, контроль сроков.

3.2. Диаграмма прецедентов

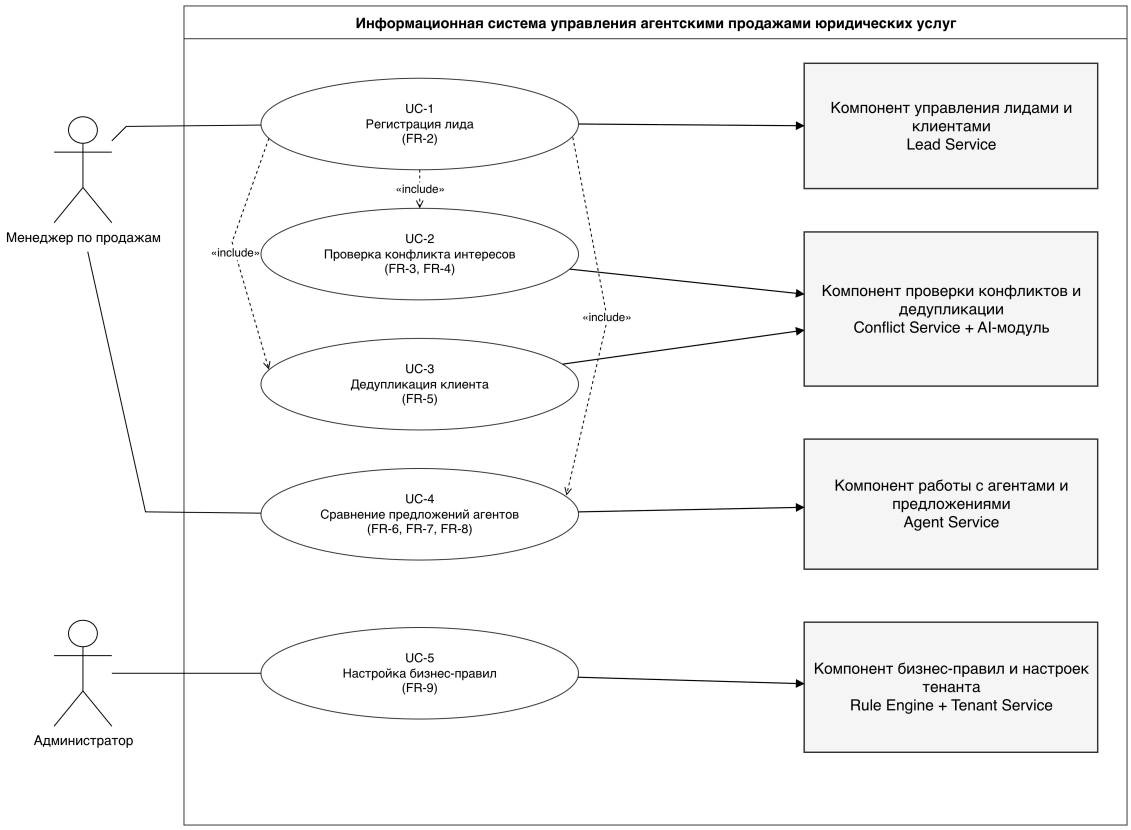


Рис. 3.1: Диаграмма прецедентов системы

3.3. Описание основных прецедентов

UC-1: Регистрация лида

- **Актор:** Менеджер по продажам.
- **Предусловие:** Пользователь авторизован в системе.
- **Основной поток:**

  1. Пользователь открывает форму регистрации лида.
  2. Вводит данные клиента, агента, оппонента и финансовые условия.
  3. Система запускает проверку конфликта интересов (UC-2).
  4. Система выполняет дедупликацию клиента (UC-3).
  5. Система регистрирует лид и присваивает ему уникальный идентификатор.

6. Система запускает таймер приоритета агента.

- **Постусловие:** Лид сохранён в базе данных организации.

UC-2: Проверка конфликта интересов

- **Актор:** Система (запускается при регистрации лида).
- **Предусловие:** В системе указан оппонент по делу.
- **Основной поток:**
  1. Система извлекает название компании-оппонента.
  2. Применяет NLP-нормализацию (UC-11).
  3. Проверяет вхождение компании в базу текущих клиентов организации.
  4. Проверяет вхождение в базу архивных клиентов с учётом срока давности.
  5. Проверяет аффилированность по глобальному справочнику (UC-10).
  6. Формирует отчёт о наличии/отсутствии конфликта.
- **Постусловие:** Менеджер получает уведомление о результате проверки.

UC-3: Дедупликация клиента

- **Актор:** Система, Менеджер по продажам.
- **Предусловие:** В систему введены данные нового клиента.
- **Основной поток:**
  1. Система получает данные клиента, указанные при регистрации лида.
  2. Выполняет поиск точных совпадений по идентифицирующим данным (телефон, e-mail, ИНН).

3. Выполняет поиск возможных дубликатов по сходным данным (ФИО или название организации, адрес и другие реквизиты).
4. Система рассчитывает степень сходства найденных записей.
5. При обнаружении вероятного дубликата система отображает менеджеру существующую запись клиента и найденные совпадения.
6. Менеджер подтверждает использование существующей записи либо создание нового клиента.

- **Постусловие:** Лид связан с существующим клиентом либо в системе создана новая уникальная запись клиента.

UC-4: Сравнение предложений агентов

- **Актор:** Менеджер по продажам.
- **Предусловие:** Для одного клиента зарегистрированы предложения от двух или более агентов.
- **Основной поток:**

1. Система определяет все активные предложения агентов, относящиеся к одному клиенту.
2. Для каждого предложения система получает сумму сделки и размер комиссии агента.
3. Система рассчитывает ожидаемую чистую прибыль юридической фирмы по каждому предложению.
4. Система проверяет действительность приоритета каждого агента с учётом установленного срока.
5. Менеджеру отображается сводная таблица предложений, отсортированная по ожидаемой чистой прибыли.
6. Система выделяет наиболее выгодное предложение.
7. Менеджер выбирает предложение для дальнейшей работы.

- **Постусловие:** Выбранное предложение зафиксировано для лида, решение менеджера сохранено в системе.

UC-5: Настройка бизнес-правил

- **Актор:** Администратор организации.
- **Предусловие:** Пользователь имеет роль администратора организации.
- **Основной поток:**
  1. Администратор открывает конструктор правил.
  2. Выбирает тип правила (конфликт интересов, приоритет агента и т.д.).
  3. Визуально формирует условие (например: «ЕСЛИ оппонент = Сбербанк, ТО срок проверки = 10 лет»).
  4. Сохраняет правило в базе данных с привязкой к локальной системе организации.
- **Постусловие:** Правило применено ко всем будущим операциям внутри системы этой организации.

4. Архитектура системы

4.1. Общая архитектура

Система построена по трёхуровневой клиент-серверной архитектуре и предназначена для развёртывания в локальной инфраструктуре юридической компании.

Пользователи работают с системой через веб-клиент, запускаемый в браузере на рабочих компьютерах сотрудников. Клиентская часть реализуется как SPA-приложение на Angular и взаимодействует с серверной частью по HTTP/HTTPS через REST API.

Серверная часть разворачивается на локальном сервере организации и реализуется как монолитное Spring Boot-приложение. Оно содержит контроллеры Spring MVC, механизмы аутентификации и авторизации Spring Security, бизнес-логику системы, движок декларативных правил и модули интеграции с внешними сервисами.

Все пользователи одной организации работают с общей базой данных PostgreSQL, расположенной в локальной инфраструктуре компании. Доступ к данным осуществляется только через серверное приложение; прямое подключение клиентских рабочих мест к базе данных не предусматривается.

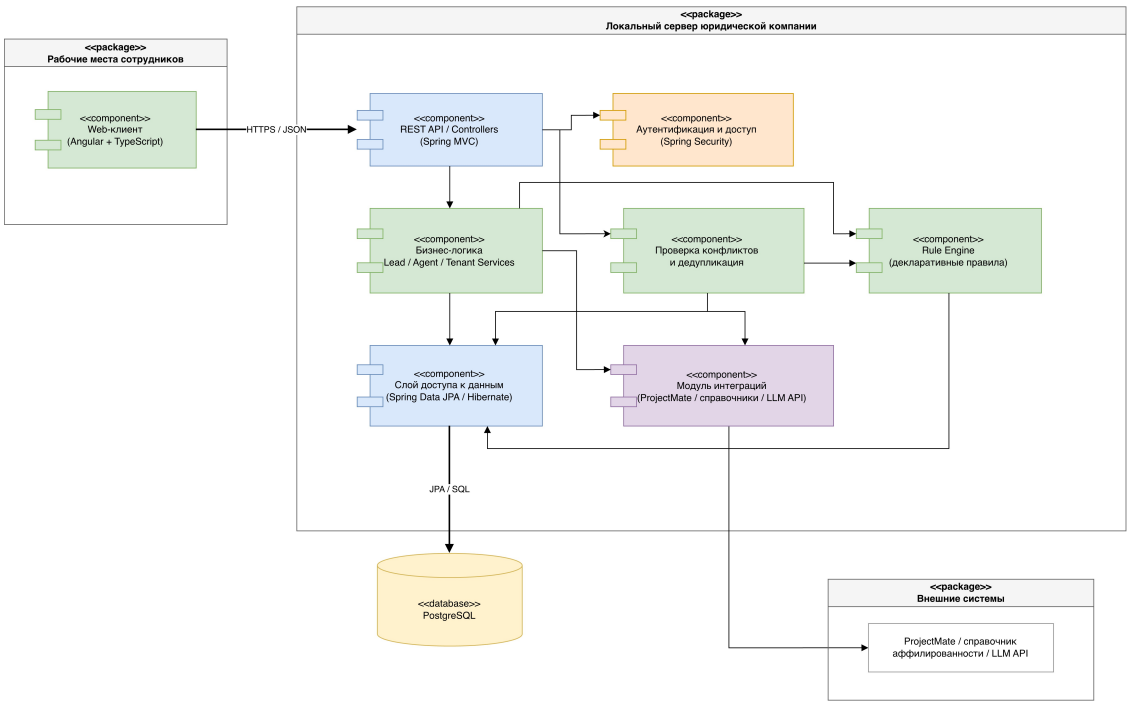


Рис. 4.1: Диаграммы компонентов системы

4.2. Ключевые компоненты

1. **Клиентское приложение.** Веб-приложение на Angular, запускаемое в браузере пользователя. Предоставляет интерфейс менеджерам и администраторам для регистрации лидов, работы с клиентами и агентами, просмотра результатов проверок и настройки бизнес-правил.

2. **REST API.** Реализуется средствами Spring MVC и является единой точкой взаимодействия клиентского приложения с серверной частью. Контроллеры принимают HTTP-запросы, выполняют проверку входных данных и передают выполнение соответствующим компонентам бизнес-логики.

3. **Модуль безопасности.** Реализуется на Spring Security и отвечает за аутентификацию пользователей, авторизацию и разграничение доступа в зависимости от роли пользователя.

4. **Прикладной уровень.** Содержит основную бизнес-логику информационной системы:

    - управление клиентами и лидами;

- управление агентами и их приоритетами;
- проверку конфликтов интересов;
- выявление возможных дубликатов клиентов;
- сравнение предложений агентов и расчёт чистой прибыли.

5. **Rule Engine.** Внутренний компонент серверного приложения, предназначенный для хранения и выполнения декларативных бизнес-правил. Правила могут изменяться администратором без изменения исходного кода приложения.

6. **Модуль интеграций.** Инкапсулирует взаимодействие с внешними системами и справочниками. Через данный компонент осуществляется интеграция с ProjectMate, источниками информации об аффилированности организаций и, при необходимости, внешним LLM API для нормализации названий и поиска похожих записей.

7. **Слой доступа к данным.** Реализуется с использованием Spring Data JPA и Hibernate. Компонент обеспечивает сохранение и получение данных бизнес-объектов, выполнение транзакционных операций и взаимодействие с PostgreSQL.

8. **База данных.** PostgreSQL используется как единое централизованное хранилище данных юридической компании. В базе хранятся сведения о клиентах, лидах, агентах, предложениях, бизнес-правилах и результатах выполненных проверок.

4.3. Развёртывание системы

Серверная часть приложения и PostgreSQL размещаются в локальной инфраструктуре юридической компании. Пользовательские рабочие места находятся в одной корпоративной сети и обращаются к серверному приложению через браузер.

Такой подход позволяет централизованно хранить и обрабатывать конфиденциальные данные клиентов, не размещая основную базу данных на пользовательских компьютерах. Обновление серверной части выполня-

ется централизованно и не требует установки новой версии приложения на каждом рабочем месте.

Внешние сервисы не получают прямого доступа к базе данных. Обмен данными с ними выполняется только через модуль интеграций серверного приложения.

5. Технологический стек

Уровень	**Технология**
Frontend	Angular + TypeScript. Angular предоставляет встроенные средства маршрутизации, работы с формами и HTTP-взаимодействия с серверной частью.
Backend	Spring Boot 3 + Spring MVC + Spring Data JPA. Обоснование: соответствует требованию задания (Spring MVC), обеспечивает быструю разработку REST API, встроенная поддержка транзакций и безопасности.
База данных	PostgreSQL 15. Обоснование: соответствует требованию задания, поддержка JSONB для хранения декларативных правил.
Rule Engine	Собственная реализация на базе JSON-схем и интерпретатора условий.
AI	Интеграция с внешним LLM API (например, YandexGPT или open-source модель, развёрнутая локально) для нормализации названий и нечёткого поиска.
Сборка и развёртывание	Gradle (backend), npm (frontend).
Контроль версий	Git

Заключение по этапу 1

В рамках первого этапа курсовой работы проведён детальный анализ предметной области управления агентскими продажами юридических услуг. Выявлены ключевые проблемы, связанные с ручной проверкой конфликтов интересов, дублированием лидов, отсутствием учёта приоритетов агентов и невозможностью гибкой настройки бизнес-правил.

Сформулированы функциональные и нефункциональные требования к разрабатываемой информационной системе, построены модели прецедентов с описанием основных сценариев использования. Предложена двухуровневая архитектура системы с декларативным движком бизнес-правил.

Выбранный технологический стек (Angular + Spring Boot + PostgreSQL) соответствует требованиям задания и может быть развёрнут на сервере helios.

Следующим этапом работы будет детальное проектирование базы данных (ER-диаграмма), разработка API-контрактов и реализация прототипа ключевых модулей системы.